```
"""
Narrative Sensitivity Detector using Local LLM (Ollama + Llama 3.1 8B)

This module analyzes text for contextual sensitivity factors that
rule-based PII/QI detectors miss: inferential risks, narrative
uniqueness, social context, and other contextual sensitivity cues.
"""

import json
import requests
from dataclasses import dataclass, field
from typing import Optional
from enum import Enum
from pathlib import Path

from example_inputs import load_example_inputs


class RiskLevel(Enum):
    LOW = "LOW"
    MEDIUM = "MEDIUM"
    HIGH = "HIGH"
    CRITICAL = "CRITICAL"


@dataclass
class SensitivityFactor:
    """A single sensitivity factor detected by the LLM."""
    name: str
    detected: bool
    confidence: float  # 0.0 to 1.0
    explanation: str


@dataclass
class NarrativeAnalysis:
    """Complete analysis result from the narrative detector."""
    factors: list[SensitivityFactor]
    overall_risk: RiskLevel
    risk_score: float  # 0.0 to 1.0
    summary: str
    raw_llm_response: str

    def __str__(self):
        lines = [
            "=" * 70,
            "NARRATIVE SENSITIVITY ANALYSIS",
            "=" * 70,
            f"Overall Risk: {self.overall_risk.value}",
            f"Risk Score: {self.risk_score:.2f}",
            "",
            "Factors Detected:",
            "-" * 40,
        ]

        for factor in self.factors:
            status = "✓ YES" if factor.detected else "✗ NO"
            lines.append(f"  {factor.name}:")
            lines.append(f"    Status: {status} (confidence:
{factor.confidence:.2f})")
            if factor.detected:
                lines.append(f"    Reason: {factor.explanation}")

        lines.extend([
```

```
            "",
            "-" * 40,
            "Summary:",
            self.summary,
            "=" * 70,
        ])

        return "\n".join(lines)


class NarrativeSensitivityDetector:
    """
    Detects contextual sensitivity in narrative text using a local LLM.

    This complements rule-based PII/QI detection by identifying:
    - Inferential disclosure risks
    - Narrative uniqueness
    - Social/employment sensitivity
    - Mental health indicators
    - Stigma and discrimination risks
    - Small community risks
    - Temporal correlation risks
    """

    SENSITIVITY_FACTORS = [
        {
            "name": "INFERENTIAL_DISCLOSURE",
            "description": "Sensitive facts can be inferred even if explicit
identifiers were removed",
            "examples": "Disease + symptoms + treatment revealing condition;
lifestyle details revealing identity"
        },
        {
            "name": "NARRATIVE_UNIQUENESS",
            "description": "The narrative describes events or circumstances
unusual enough to identify the person",
            "examples": "Unique accidents, rare achievements, unusual life
circumstances, memorable public events"
        },
        {
            "name": "EMPLOYMENT_SENSITIVITY",
            "description": "Employment-related concerns that could harm the
individual if disclosed",
            "examples": "Job performance issues, workplace conflicts, concerns
about accommodation requests"
        },
        {
            "name": "MENTAL_HEALTH_INDICATORS",
            "description": "Mental health information requiring special
protection",
            "examples": "Anxiety, depression, therapy mentions, psychiatric
medications, emotional distress"
        },
        {
            "name": "SOCIAL_STIGMA_RISK",
            "description": "Disclosure could lead to social stigma,
discrimination, or embarrassment",
            "examples": "Stigmatized conditions, embarrassing incidents,
socially sensitive behaviors"
        },
        {
            "name": "SMALL_COMMUNITY_RISK",
            "description": "Set in a small community where individuals are
easily identifiable",
```

```
            "examples": "Small towns, tight-knit workplaces, niche communities,
small schools"
        },
        {
            "name": "TEMPORAL_CORRELATION_RISK",
            "description": "Timestamps could be correlated with external records
to identify the person",
            "examples": "Specific dates of events, sequences that match records,
appointment times"
        },
    ]

    def __init__(
        self,
        model: str = "qwen2.5:3b",
        ollama_url: str = "http://localhost:11434",
        temperature: float = 0.1,  # Low temperature for consistent analysis
        timeout: int = 120
    ):
        self.model = model
        self.ollama_url = ollama_url
        self.temperature = temperature
        self.timeout = timeout

    def _build_prompt(self, text: str) -> str:
        """Build the analysis prompt with clear examples."""

        factors_description = "\n".join([
            f"- {f['name']}: {f['description']}"
            for f in self.SENSITIVITY_FACTORS
        ])

        prompt = f"""You are a privacy analyst evaluating ALREADY-REDACTED text
for residual risks.

    CRITICAL RULE: Anything in [BRACKETS] like [PERSON], [DISEASE],
[ORGANIZATION] is a REDACTION PLACEHOLDER.
    The actual sensitive data has been REMOVED. Do NOT treat placeholders as
sensitive information, as they are unknown.

    YOUR JOB: Find risks in the UNREDACTED parts of the text.

    === EXAMPLE: LOW RISK ===
    Text: "[PERSON] visited [ORGANIZATION] for treatment of [DISEASE]. Dr.
[PERSON] provided a prescription."
    Analysis: LOW RISK - All specifics are redacted. Generic medical visit
structure reveals nothing.
    Factors detected: None

    === EXAMPLE: HIGH RISK ===
    Text: "[PERSON] was the sole survivor of the Flight 447 crash and is being
treated at [ORGANIZATION]."
    Analysis: HIGH RISK - "sole survivor of Flight 447 crash" is unique enough
to identify via news search.
    Factors detected: NARRATIVE_UNIQUENESS (high), INFERENTIAL_DISCLOSURE
(medium)

    === ANALYZE THIS TEXT ===
    \"\"\"{text}\"\"\"

    FACTORS TO EVALUATE:
    {factors_description}

    For each factor, assess the UNREDACTED content only. Attributes in square
```

```
brackets like [PERSON] are NOT evidence of risk.

    Return JSON:
    {{
        "factors": [
            {{"name": "INFERENTIAL_DISCLOSURE", "detected": false, "confidence":
0.0, "explanation": ""}},
            {{"name": "NARRATIVE_UNIQUENESS", "detected": false, "confidence":
0.0, "explanation": ""}},
            {{"name": "EMPLOYMENT_SENSITIVITY", "detected": false, "confidence":
0.0, "explanation": ""}},
            {{"name": "MENTAL_HEALTH_INDICATORS", "detected": false,
"confidence": 0.0, "explanation": ""}},
            {{"name": "SOCIAL_STIGMA_RISK", "detected": false, "confidence":
0.0, "explanation": ""}},
            {{"name": "SMALL_COMMUNITY_RISK", "detected": false, "confidence":
0.0, "explanation": ""}},
            {{"name": "TEMPORAL_CORRELATION_RISK", "detected": false,
"confidence": 0.0, "explanation": ""}}
        ],
        "summary": "Assessment based on unredacted content"
    }}"""

        return prompt

    def _call_ollama(self, prompt: str) -> str:
        """Make a request to the Ollama API."""

        url = f"{self.ollama_url}/api/generate"

        payload = {
            "model": self.model,
            "prompt": prompt,
            "temperature": self.temperature,
            "stream": False,
            "format": "json"  # Request JSON format
        }

        try:
            response = requests.post(
                url,
                json=payload,
                timeout=self.timeout
            )
            response.raise_for_status()

            result = response.json()
            return result.get("response", "")

        except requests.exceptions.ConnectionError:
            raise ConnectionError(
                "Could not connect to Ollama. Make sure it's running with:
ollama serve"
            )
        except requests.exceptions.Timeout:
            raise TimeoutError(
                f"Ollama request timed out after {self.timeout} seconds"
            )

    def _parse_response(self, response: str) -> dict:
        """Parse the JSON response from the LLM."""

        # Try to extract JSON if there's extra text
        response = response.strip()
```

```
        # Find JSON boundaries if needed
        if not response.startswith("{"):
            start = response.find("{")
            if start != -1:
                response = response[start:]

        if not response.endswith("}"):
            end = response.rfind("}")
            if end != -1:
                response = response[:end + 1]

        try:
            return json.loads(response)
        except json.JSONDecodeError as e:
            # Return a default structure on parse failure
            return {
                "factors": [],
                "summary": f"Failed to parse LLM response: {str(e)}",
                "parse_error": True
            }

    def _calculate_risk_score(self, factors: list[SensitivityFactor]) ->
tuple[float, RiskLevel]:
        """Calculate overall risk score from individual factors."""

        # Weights for each factor (sum to ~1.0 for normalization)
        weights = {
            "INFERENTIAL_DISCLOSURE": 0.20,
            "NARRATIVE_UNIQUENESS": 0.18,
            "EMPLOYMENT_SENSITIVITY": 0.12,
            "MENTAL_HEALTH_INDICATORS": 0.12,
            "SOCIAL_STIGMA_RISK": 0.14,
            "SMALL_COMMUNITY_RISK": 0.14,
            "TEMPORAL_CORRELATION_RISK": 0.10,
        }

        total_score = 0.0

        for factor in factors:
            if factor.detected:
                weight = weights.get(factor.name, 0.1)
                # Score contribution = weight × confidence
                total_score += weight * factor.confidence

        # Normalize to 0-1 range
        risk_score = min(total_score, 1.0)

        # Determine risk level
        if risk_score >= 0.6:
            risk_level = RiskLevel.CRITICAL
        elif risk_score >= 0.4:
            risk_level = RiskLevel.HIGH
        elif risk_score >= 0.2:
            risk_level = RiskLevel.MEDIUM
        else:
            risk_level = RiskLevel.LOW

        return risk_score, risk_level

    def analyze(self, text: str) -> NarrativeAnalysis:
        """
        Analyze text for contextual sensitivity factors.
```

```
        Args:
            text: The narrative text to analyze

        Returns:
            NarrativeAnalysis object with detailed results
        """

        # Build and send prompt
        prompt = self._build_prompt(text)
        raw_response = self._call_ollama(prompt)

        # Parse response
        parsed = self._parse_response(raw_response)

        # Build factor objects
        factors = []
        for f_data in parsed.get("factors", []):
            factor = SensitivityFactor(
                name=f_data.get("name", "UNKNOWN"),
                detected=f_data.get("detected", False),
                confidence=float(f_data.get("confidence", 0.0)),
                explanation=f_data.get("explanation", "")
            )
            factors.append(factor)

        # Ensure we have all factors (fill in missing ones)
        found_names = {f.name for f in factors}
        for expected in self.SENSITIVITY_FACTORS:
            if expected["name"] not in found_names:
                factors.append(SensitivityFactor(
                    name=expected["name"],
                    detected=False,
                    confidence=0.0,
                    explanation=""
                ))

        # Sort factors by name for consistent ordering
        factors.sort(key=lambda f: f.name)

        # Calculate risk
        risk_score, risk_level = self._calculate_risk_score(factors)

        # Get summary
        summary = parsed.get("summary", "Analysis complete.")

        return NarrativeAnalysis(
            factors=factors,
            overall_risk=risk_level,
            risk_score=risk_score,
            summary=summary,
            raw_llm_response=raw_response
        )


def run_tests():
    """Run the detector against all test cases."""

    print("=" * 70)
    print("NARRATIVE SENSITIVITY DETECTOR - TEST SUITE")
    print("Model: Llama 3.2 3B via Ollama")
    print("=" * 70)
    print()

    detector = NarrativeSensitivityDetector()
```

```
    examples = load_example_inputs()

    results = []

    for i, test in enumerate(examples, 1):
        name = test.get("name", f"Example {i}")
        print(f"\n{'=' * 70}")
        print(f"TEST {i}: {name}")
        print(f"{'=' * 70}")
        print(f"\nInput Text:\n{test['text'][:200]}...")
        print("\nAnalyzing...")

        try:
            analysis = detector.analyze(test["text"])
            print(analysis)

            detections = []
            for f in analysis.factors:
                detections.append({
                    "type": f.name,
                    "detected": f.detected,
                    "confidence": f.confidence,
                    "explanation": f.explanation,
                    "start": None,
                    "end": None,
                    "text": None,
                })

            results.append({
                "name": name,
                "text": test["text"],
                "overall_risk": analysis.overall_risk.value,
                "risk_score": analysis.risk_score,
                "detections": detections,
            })

        except Exception as e:
            print(f"\n✗ ERROR: {str(e)}")
            results.append({
                "name": name,
                "text": test.get("text", ""),
                "error": str(e),
                "detections": [],
            })

    print("\n" + "=" * 70)
    print("TEST COMPLETE")
    print("=" * 70)
    out_dir = Path("outputs")
    out_dir.mkdir(exist_ok=True)
    out_file = out_dir / "c_score_results.json"
    out_file.write_text(json.dumps(results, indent=2, ensure_ascii=False))
    print(f"Unified JSON results written to: {out_file}")


if __name__ == "__main__":
    # Put your single input text here
    text_to_analyze = """
    Patient [PERSON] was seen at Boston Medical Center.\n    Dr. [PERSON]
prescribed medication for [DISEASE].\n    Contact: [URL]ADDRESS]HONE_NUMBER].
[ORGANIZATION]: [US_SSN]
    """

    detector = NarrativeSensitivityDetector()
```

```
try:
    analysis = detector.analyze(text_to_analyze)
    print(analysis)

    # Optional: save result to file
    out_dir = Path("outputs")
    out_dir.mkdir(exist_ok=True)
    out_file = out_dir / "single_analysis_result.json"

    result = {
        "text": text_to_analyze,
        "overall_risk": analysis.overall_risk.value,
        "risk_score": analysis.risk_score,
        "summary": analysis.summary,
        "detections": [
            {
                "type": f.name,
                "detected": f.detected,
                "confidence": f.confidence,
                "explanation": f.explanation,
            }
            for f in analysis.factors
        ],
        "raw_llm_response": analysis.raw_llm_response,
    }

    out_file.write_text(json.dumps(result, indent=2, ensure_ascii=False))
    print(f"\nResult written to: {out_file}")

except Exception as e:
    print(f"ERROR: {str(e)}")
```